

1.3 Listen - Lösung

Listen haben den gleichen Grundaufbau wie **Queues** – Es wird nur eine Referenz auf den Kopf gehalten und die einzelnen Knoten sind als **rekursive Datenstruktur** miteinander **verkettet**. Die Liste kann jedoch mehr - so kann an beliebiger Stelle eingefügt, entfernt oder auslesen werden.

Hierzu erweitern wir außerdem die Funktionen der Klasse **Node**

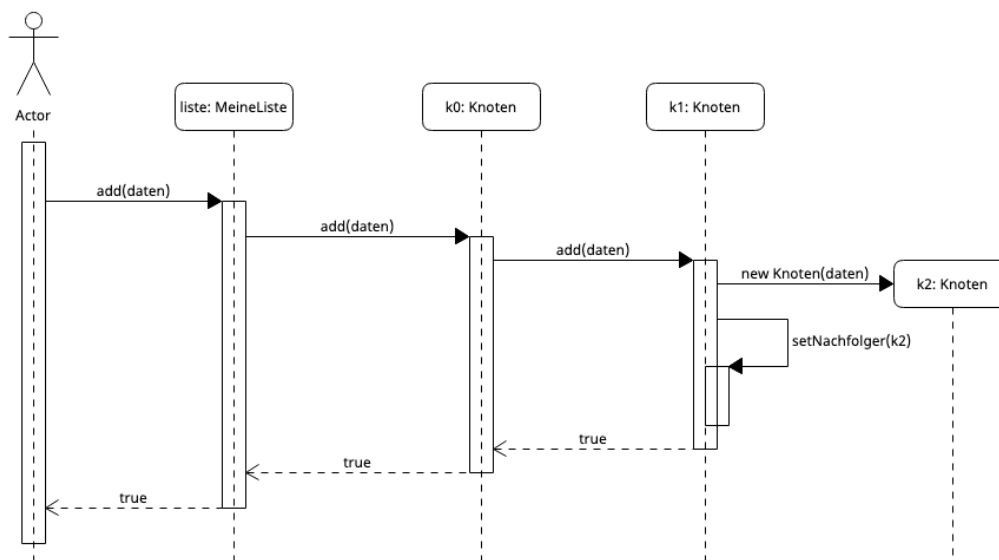
Einfügen am Datenelement durch Durchreichen

```
public boolean add(DataElement data) {  
    if(successor == null) {  
        successor = new Node(data);  
        return true;  
    } else {  
        return successor.add(data); // Selbstauf Ruf --> Rekursion  
    }  
}
```

Aufruf der gleichen Methode innerhalb der Methode führt zu einer Verschachtelung -> **Rekursiver Aufruf**. Die Methode gibt einfach dem nächsten Knoten den gleichen Auftrag weiter.

Von **Rekursion** spricht man, wenn innerhalb einer Methode die Methode selbst erneut aufgerufen wird. Dies läuft theoretisch **unendlich**, deshalb muss eine **Abbruchbedingung** eingeführt werden, welche dann greift, wenn die Methode erfolgreich umgesetzt werden kann (hier **am passenden bzw. letzten Knoten**) oder die Bedingung nicht mehr erfüllt werden kann (z.B. festgelegte Länge überschritten).

Die Verschachtelung ist auch sehr gut im **UML-Sequenzdiagramm** zu erkennen:



Lebenslinien, Akteure, Objekte; Aktivitätsbalken; synchroner Aufruf / Nachricht; Rückgaben / Antwort;

Aufgabe 1: Benennen Sie die Elemente im Sequenzdiagramm mit Fachbegriffen.

Aufgabe 2)

- Duplizieren Sie ihren Workspace für die Taxiqueue und verändern Sie die Namen mittels Rechtsklick „Rename Symbol“ und passen Sie das Projekt so an, dass es eine Liste umsetzt (nur Referenz auf Anfang).
- Übertragen Sie den Programmcode für die Methode `add(DatenElement daten)` in Ihr Projekt und adaptieren Sie entsprechend die Methode in der Klasse `MeineListe`.
- Ergänzen Sie die Methoden `länge()` und `entfernen(DatenElement daten)` in `MeineListe` und `Knoten` mit der entsprechenden Funktionalität. Übertrage den Code unten.

class `MeineListe`

```
int Länge() {
    if(anfang == null) {
        //leere Liste
        return 0;
    } else {
        //Rekursion
        return anfang.Länge();
    }
}
```

```
boolean entfernen(DatenElement daten) {
    //Liste nicht leer
    if(anfang != null) {
        // speichere anfang zwischen
        if(anfang.getData() == daten) {
            anfang = anfang.getNachfolger();
            return true;
        } else {
            //Rekursion
            return anfang.entfernen(daten);
        }
    } else {
        return false;
    }
}
```

class `Knoten`:

```
int Länge() {
    if(nachfolger == null) {
        // Ende der Liste
        return 1;
    } else {
        // Rekursion
        return nachfolger.Länge() + 1;
    }
}
```

```
boolean entfernen(DatenElement daten) {
    //gibt es noch einen Nachfolger
    if(nachfolger != null) {
        //hat mein Nachfolger die passenden Daten?
        if(nachfolger.getData() == daten) {
            //ja, dann überspringe ich ihn und hole mir seinen Nachfolger
            nachfolger = nachfolger.getNachfolger();
            // und war erfolgreich
            return true;
        } else {
            // Rekursion --> weitergeben der Aufgabe
            return nachfolger.entfernen(daten);
        }
    } else {
        //kein Nachfolger aber nicht gefunden --> falsch
        return false;
    }
}
```

Aufgabe 3: Zeichnen Sie das Sequenzdiagramm zu Aufruf von `länge()` auf einem Objekt `liste` der Klasse `MeineListe`, welche drei `Knoten` umfasst.

